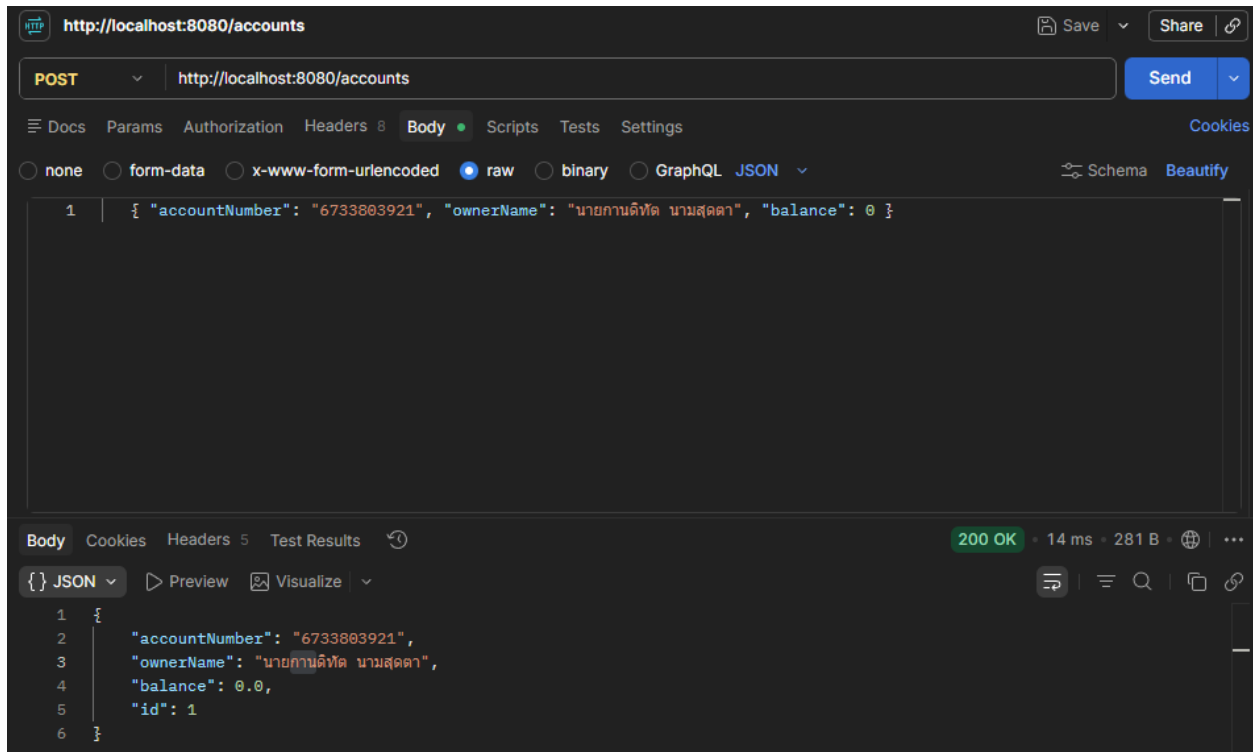


673380392-1 นายกานดิทัต นามสุดตา Sec 3.

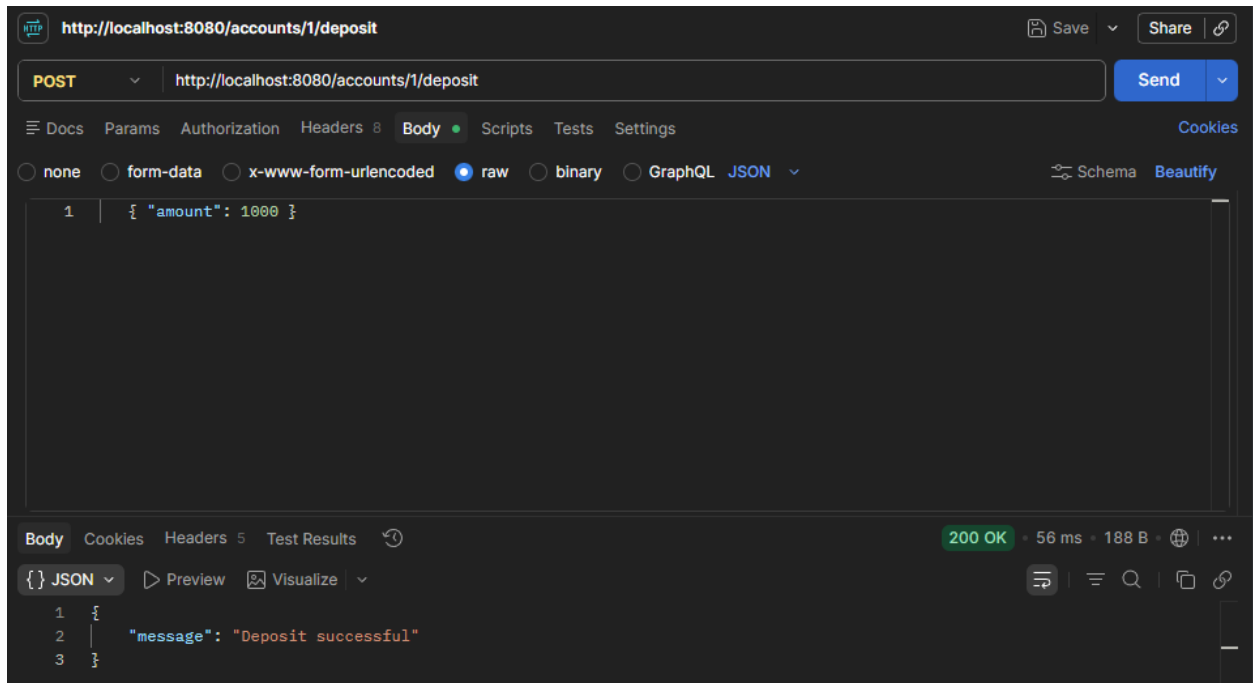
1. สร้าง Account



อธิบายผล: ส่ง Request แบบ POST /accounts พร้อมข้อมูล accountNumber, ownerName และ balance เริ่มต้นเป็น 0 ระบบตอบกลับด้วย Status 200 OK และคืนค่าข้อมูล Account ที่สร้างสำเร็จ พร้อม id ที่ระบบสร้างขึ้นอัตโนมัติ (id = 1) แสดงว่า Endpoint สำหรับสร้างบัญชีทำงานถูกต้อง และข้อมูลถูกบันทึกลงฐานข้อมูลจริง

673380392-1 นายกานติทัต นามสุดตา Sec 3.

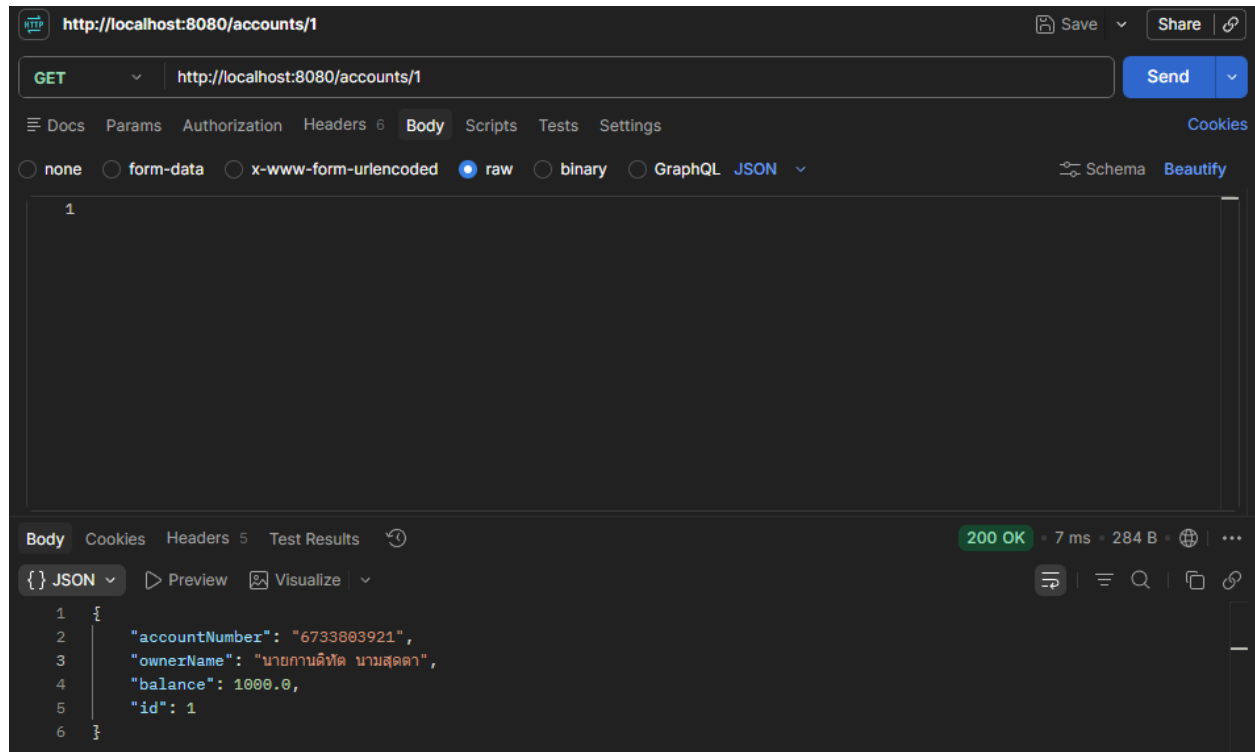
2. การฝากเงิน



อธิบายผล: ส่ง Request POST /accounts/1/deposit พร้อมจำนวนเงิน 1000 ระบบตอบกลับด้วย Status 200 OK และข้อความ "Deposit successful" แสดงว่า Service สามารถทำงานทั้งสองขั้นตอนได้สำเร็จ คือเพิ่มยอดเงินในบัญชี และบันทึกประวัติการฝากเงินลงตาราง `deposit_transaction`

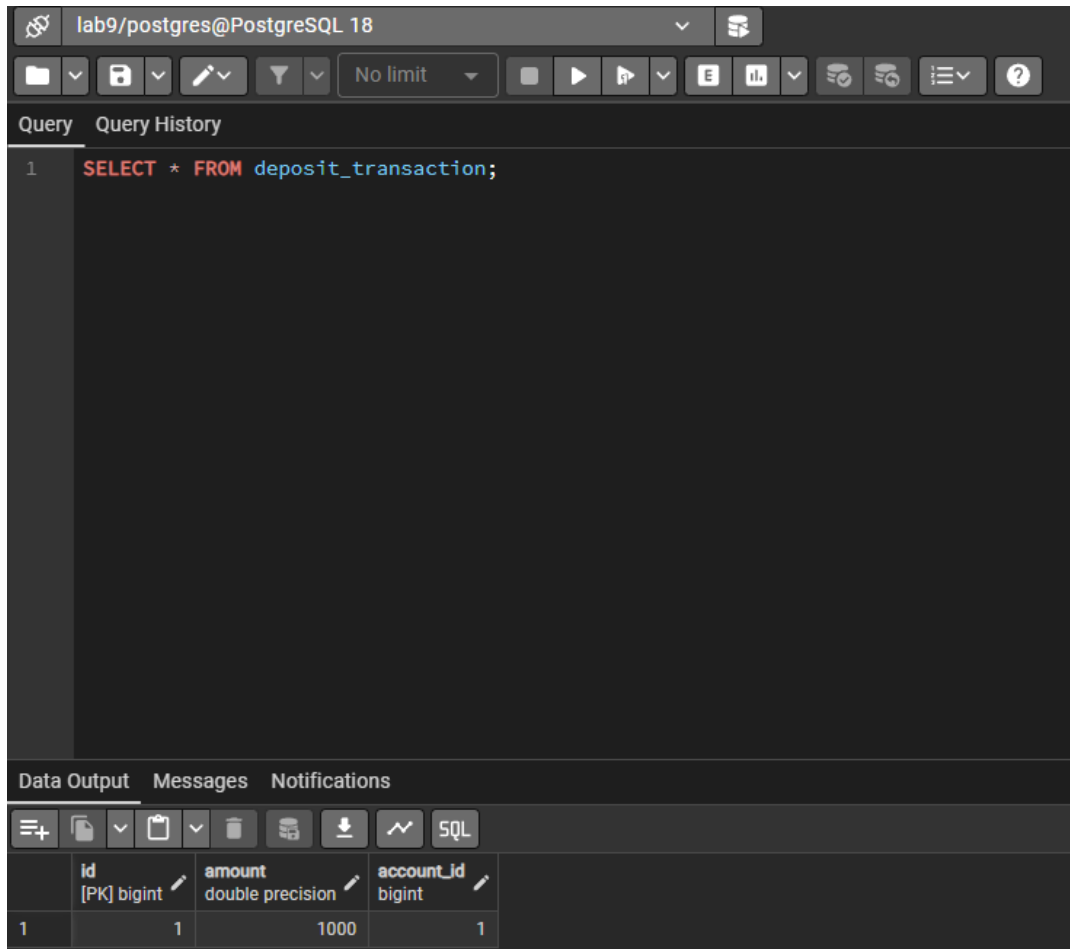
673380392-1 นายกานดิทัต นามสุดตา Sec 3.

3. ตรวจสอบ Account



อธิบายผล: หลังจากฝากเงิน เมื่อเรียก GET /accounts/1 พบว่า balance เปลี่ยนจาก 0 เป็น 1000.0
ตรงตามจำนวนเงินที่ฝากเข้าไป ยืนยันว่าขั้นตอนการอัปเดตยอดเงินในบัญชีทำงานถูกต้อง

4. ตรวจสอบ DepositTransaction



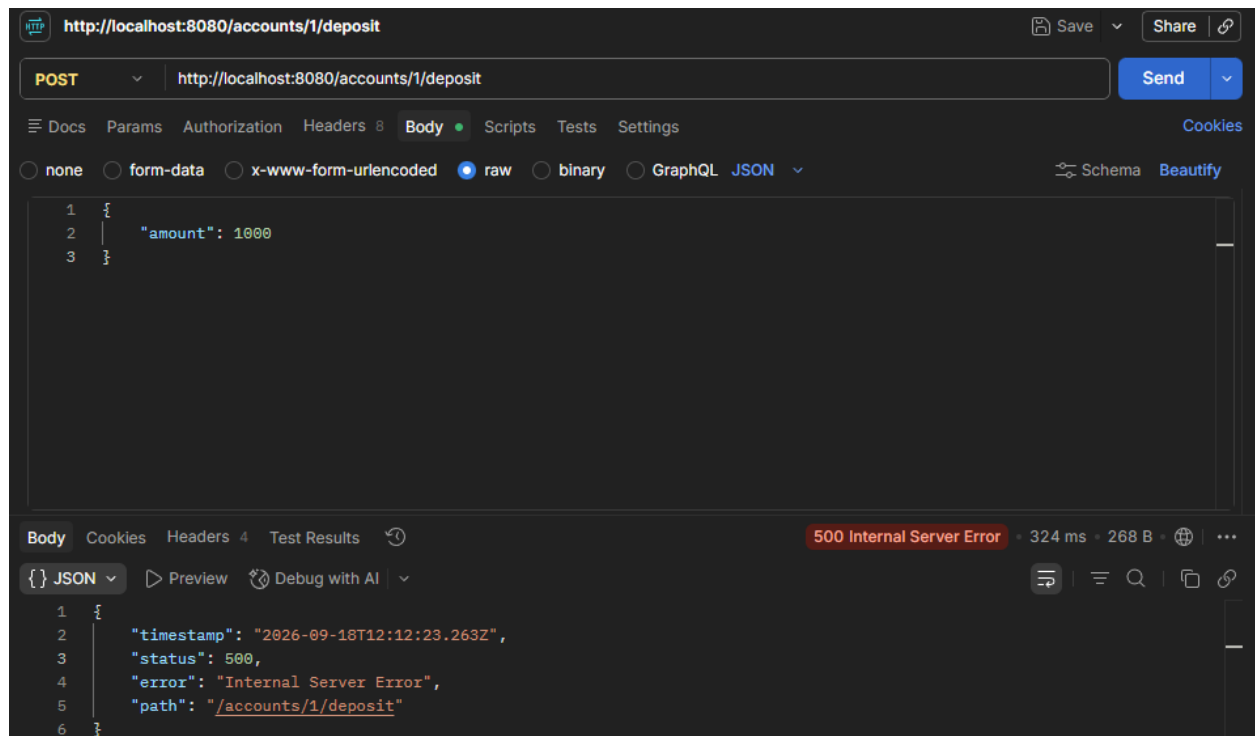
The screenshot shows the pgAdmin 4 interface. At the top, the connection is labeled 'lab9/postgres@PostgreSQL 18'. Below the toolbar, the 'Query' tab is active, displaying the SQL query: `SELECT * FROM deposit_transaction;`. The 'Data Output' tab at the bottom shows the results of the query in a table format.

	id [PK] bigint	amount double precision	account_id bigint
1	1	1000	1

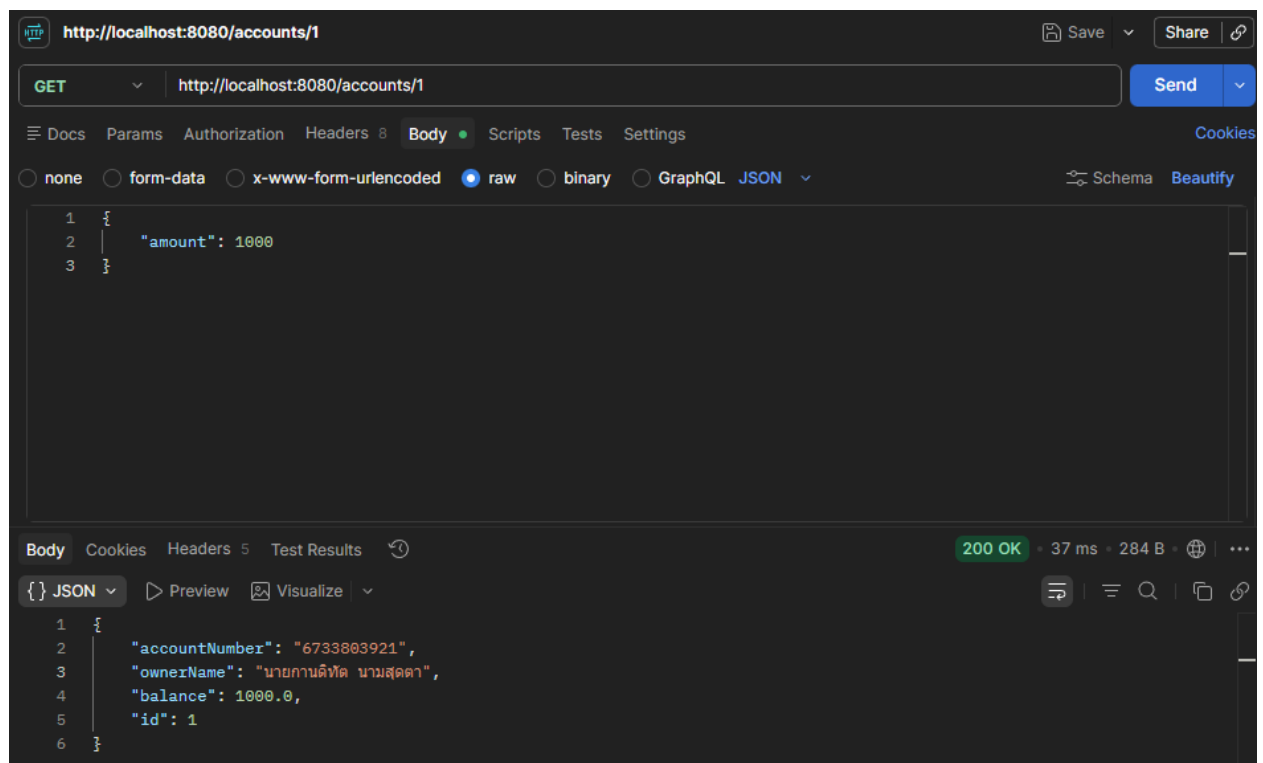
อธิบายผล: เมื่อเปิด pgAdmin แล้วรันคำสั่ง `SELECT * FROM deposit_transaction;` พบข้อมูล 1 แถว โดยมี `amount = 1000` และ `account_id = 1` ตรงกับบัญชีที่ทำการฝากเงิน แสดงว่าระบบบันทึกประวัติการฝากเงินไว้ถูกต้อง สอดคล้องกับความสัมพันธ์แบบ One-to-Many ระหว่าง Account และ DepositTransaction

673380392-1 นายกานดิทัต นามสุดตา Sec 3.

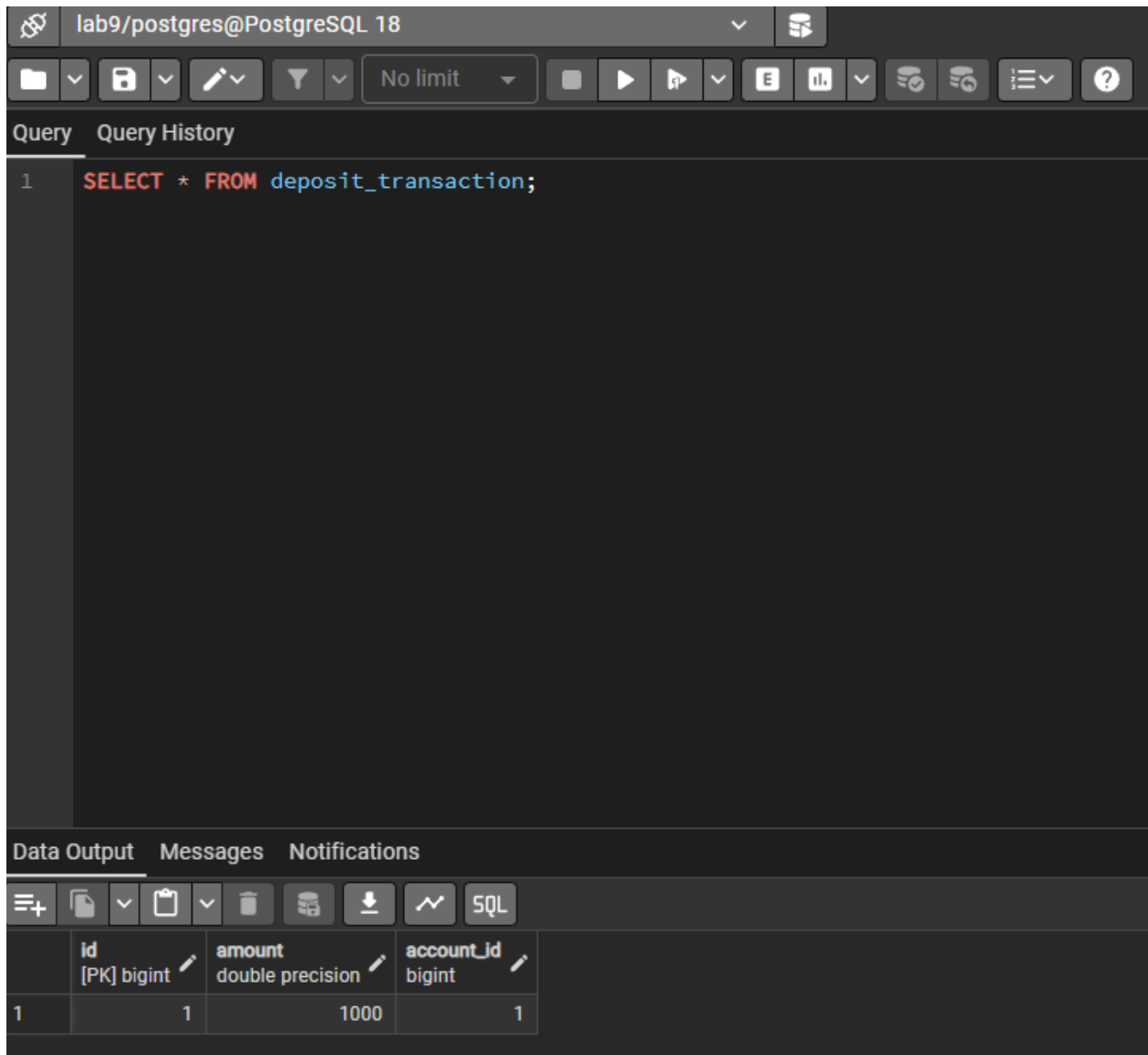
5. ทดลอง rollback



REST client interface showing a POST request to `http://localhost:8080/accounts/1/deposit`. The request body is a JSON object: `{ "amount": 1000 }`. The response is a **500 Internal Server Error** with a status of 500, a timestamp of `2026-09-18T12:12:23.263Z`, and an error message of `"Internal Server Error"`. The path is `/accounts/1/deposit`.



REST client interface showing a GET request to `http://localhost:8080/accounts/1`. The response is a **200 OK** with a status of 200, a timestamp of `2026-09-18T12:12:23.263Z`, and a JSON body containing account details: `{ "accountNumber": "6733803921", "ownerName": "นายกานดิทัต นามสุดตา", "balance": 1000.0, "id": 1 }`.



The screenshot shows a PostgreSQL query editor interface. The top bar indicates the connection is 'lab9/postgres@PostgreSQL 18'. The query editor contains the following SQL statement:

```
1 SELECT * FROM deposit_transaction;
```

The 'Data Output' tab is active, displaying the results of the query in a table format. The table has three columns: 'Id' (bigint, PK), 'amount' (double precision), and 'account_id' (bigint). The results show a single row with the following values:

	Id [PK] bigint	amount double precision	account_id bigint
1	1	1000	1

อธิบายผล: หลังจากเพิ่มโค้ด throw new RuntimeException("Test Rollback") ไว้ท้ายเมธอด deposit() มี @Transactional อยู่ แล้วยิง Request ฝากเงินเข้า ระบบตอบกลับด้วย Status 500 Internal Server Error และตรวจสอบข้อมูลอีกครั้ง balance ของบัญชียังคงเป็น 1000 เท่าเดิม และตาราง deposit_transaction มี 1 แถวเท่าเดิม แม้ว่าโค้ดจะสั่ง save() ทั้งสองคำสั่งไปแล้วก่อนเกิด Error ก็ตาม แสดงให้เห็นว่า @Transactional ทำหน้าที่ ROLLBACK ย้อนการเปลี่ยนแปลงข้อมูลทั้งหมดกลับ เมื่อเกิดข้อผิดพลาดระหว่างการทำงาน

673380392-1 นายกานดิทัต นามสุดตา Sec 3.

6. เมื่อไม่มี @Transactional

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:8080/accounts/1/deposit`
- Method:** `POST`
- Body:**

```
{
  "amount": 1000
}
```
- Response:** `500 Internal Server Error` (344 ms, 268 B)
- Response Body (JSON):**

```
{
  "timestamp": "2026-09-18T12:16:13.628Z",
  "status": 500,
  "error": "Internal Server Error",
  "path": "/accounts/1/deposit"
}
```

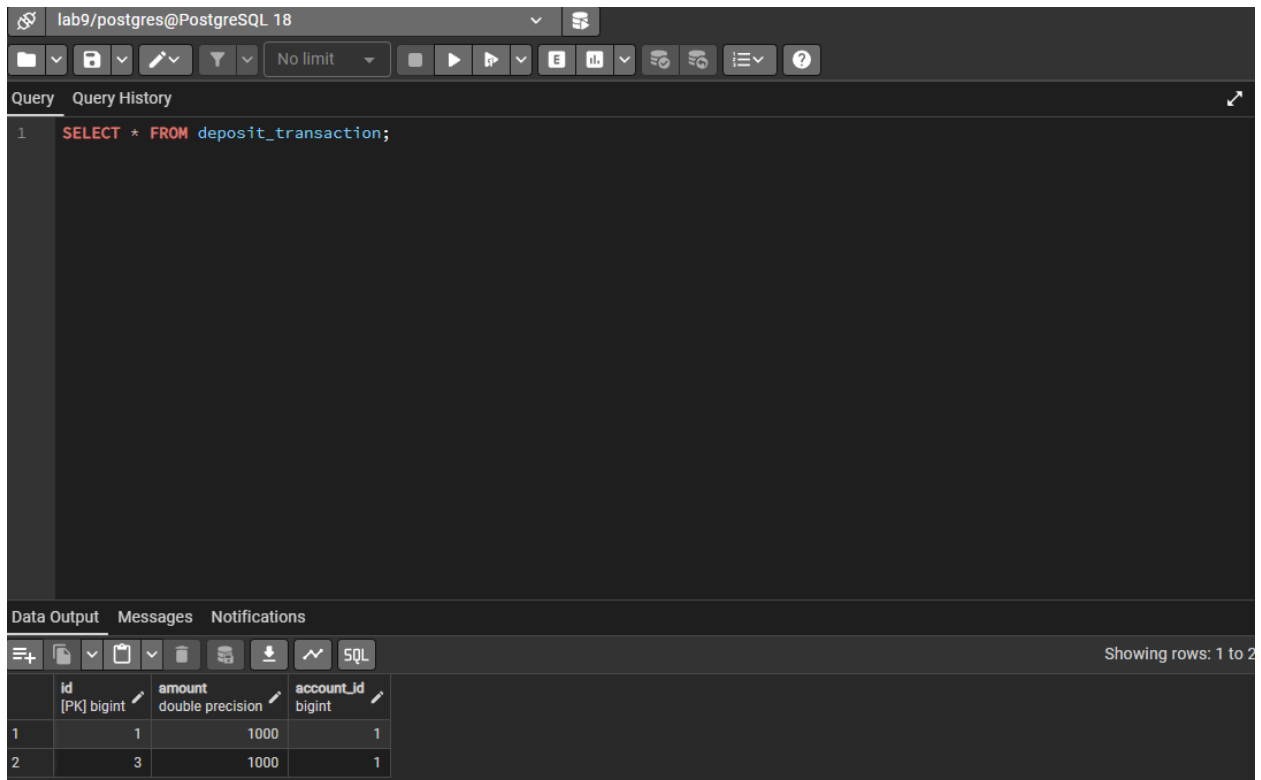
The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:8080/accounts/1`
- Method:** `GET`
- Body:**

```
{
  "amount": 1000
}
```
- Response:** `200 OK` (38 ms, 284 B)
- Response Body (JSON):**

```
{
  "accountNumber": "6733803921",
  "ownerName": "นายกานดิทัต นามสุดตา",
  "balance": 2000.0,
  "id": 1
}
```

673380392-1 นายกานดิทัต นามสุดตา Sec 3.



The screenshot shows a PostgreSQL query editor interface. The top bar indicates the connection is 'lab9/postgres@PostgreSQL 18'. The query editor contains the following SQL query:

```
1 SELECT * FROM deposit_transaction;
```

The results are displayed in the 'Data Output' tab at the bottom. The table has three columns: 'id' (PK) of type 'bigint', 'amount' of type 'double precision', and 'account_id' of type 'bigint'. There are two rows of data:

	id [PK] bigint	amount double precision	account_id bigint
1	1	1000	1
2	3	1000	1

The interface also shows 'Showing rows: 1 to 2'.

อธิบายผล: เมื่อลบ @Transactional ออกจากเมธอด deposit() โดยยังมีบรรทัด throw exception ไว้เหมือนเดิม แล้วทดลองฝากเงินซ้ำอีกครั้ง พบว่าแม้ระบบจะตอบกลับ Status 500 เช่นเดิม แต่ข้อมูลกลับไม่ถูกย้อนกลับ โดย balance ของบัญชีเพิ่มขึ้นจริง และตาราง deposit_transaction มีแถวใหม่เพิ่มขึ้นมา ทั้งที่เกิด Error หลังจากนั้น สาเหตุคือเมื่อไม่มี @Transactional แต่ละคำสั่ง save() จะถูกคอมมิตแยกจากกันทันที ไม่มีการมองภาพรวมของงานทั้งก้อน จึงไม่มีกลไกใดย้อนข้อมูลกลับไปให้เมื่อเกิดข้อผิดพลาดในภายหลัง ต่างจากกรณีที่มี @Transactional ซึ่งข้อมูลทั้งหมดถูกยกเลิกกลับไปเป็นค่าเดิม

7. ตอบคำถามท้าย Lab

- @Transactional มีหน้าที่อะไร?

ทำหน้าที่กำหนดขอบเขตของ Transaction ให้กับเมธอดนั้น โดยรวมทุกคำสั่งที่เปลี่ยนแปลงข้อมูลในเมธอดให้เป็นงานก้อนเดียวกัน ถ้าทุกคำสั่งสำเร็จจะ COMMIT ข้อมูลทั้งหมด แต่ถ้ามีคำสั่งใดล้มเหลวจะ ROLLBACK ย้อนกลับทั้งหมด

- เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

เพราะการฝากเงิน 1 ครั้งเกี่ยวข้องกับการเปลี่ยนแปลงข้อมูล 2 อย่างพร้อมกัน คือเพิ่มยอดเงินในบัญชี และบันทึกประวัติการฝาก ทั้งสองอย่างต้องสำเร็จไปด้วยกันเสมอ หากอย่างใดอย่างหนึ่งล้มเหลว อีกอย่างก็ไม่ควรถูกบันทึกไว้ ไมเช่นนั้นข้อมูลจะไม่ตรงกัน

- COMMIT และ ROLLBACK ต่างกันอย่างไร?

COMMIT คือการยืนยันบันทึกการเปลี่ยนแปลงข้อมูลทั้งหมดลงฐานข้อมูลอย่างถาวร เมื่อทุกขั้นตอนใน Transaction สำเร็จ ส่วน ROLLBACK คือการยกเลิกและย้อนการเปลี่ยนแปลงข้อมูลทั้งหมดกลับไปเป็นสถานะก่อนเริ่ม Transaction เมื่อมีข้อผิดพลาดเกิดขึ้นระหว่างทาง

- Account และ DepositTransaction มีความสัมพันธ์แบบใด?

เป็นความสัมพันธ์แบบ One-to-Many คือ 1 Account สามารถมี DepositTransaction ได้หลายรายการ โดยฝั่ง DepositTransaction จะมีฟิลด์อ้างอิงกลับไปยัง Account ด้วยความสัมพันธ์แบบ Many-to-One ผ่าน Foreign Key ชื่อ account_id

- เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

ข้อมูลจะไม่มีเปลี่ยนแปลงใดๆ เกิดขึ้นเลย แม้ว่าก่อนหน้านี้โค้ดจะส่งบันทึกข้อมูลไปแล้วหลายขั้นตอนก็ตาม เพราะระบบจะทำการ ROLLBACK ย้อนทุกอย่างกลับไปเป็นสถานะเดิมก่อนเริ่ม Transaction ทั้งหมด

- จากการทดลอง เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

ผลลัพธ์แตกต่างกันอย่างชัดเจน คือแม้จะเกิด Error เหมือนเดิม แต่ข้อมูลที่ถูกบันทึกไปก่อน

673380392-1 นายกานติทัต นามสุดตา Sec 3.

หน้าจะยังคงอยู่ในฐานข้อมูล ไม่ถูกย้อนกลับ เนื่องจากแต่ละคำสั่ง save() จะคอมมิตแยกกัน
ทันทีที่ทำงานสำเร็จ ไม่มีการรวมเป็นหน่วยงานเดียวกันเหมือนตอนมี @Transactional จึงทำ
ให้ข้อมูลในระบบไม่สอดคล้องกัน balance เพิ่มขึ้นแต่มาจาก Transaction ที่ควรจะถูกยกเลิก